GitHow

Product Requirements Document

One repository. One intelligence graph. Three ways to understand code.

Document field	Value
Version	1.0
Status	Baseline for implementation planning
Date	09 October 2026
Product	GitHow
Product modes	GitFlow · GitExplain · GitTime
Primary audience	Founding developer(s), product collaborators, AI coding agents
Related technical document	Architecture Design Document: Repository Intelligence Platform

How to use this document

This PRD defines user problems, product behavior, scope, priorities, acceptance criteria, and success measures. The Architecture Design Document remains the source of truth for implementation structure and technical choices. If a product requirement conflicts with a technical detail, preserve the user-facing requirement and update the architecture deliberately rather than silently weakening the product.

1. Executive summary

GitHow helps developers understand unfamiliar or complex codebases by turning a GitHub repository into a navigable, source-grounded model of how the software is structured, how behavior moves through it, and how that structure changes over time.

The platform analyzes a repository at a specific commit and builds a shared Repository Intelligence Graph (RIG). GitFlow, GitExplain, and GitTime are product views over that shared graph—not separate products or independent analysis pipelines.

Mode	User question	Core output
GitFlow	"How does this request, feature, or action move through the code?"	Ordered, interactive execution flow with source-linked steps and branches.
GitExplain	"What does this function/file/class do, and how is it connected?"	Grounded explanation, callers/callees, dependencies, data interactions, and evidence.
GitTime	"How did this codebase and its architecture evolve?"	Commit-aware timeline, structural diffs, entity changes, and change explanations.

The product's defining entry experience is URL-driven: a developer viewing a public repository on GitHub can replace the GitHub domain with GitHow's domain and land directly on that repository's analysis. The website's paste-a-URL flow is a secondary entry point and fallback.

2. Product vision and principles

Vision: make understanding a codebase feel as immediate as opening the codebase itself.

- URL-first access: no account or onboarding barrier for public repositories.
- Analyze once, reuse everywhere: every mode consumes the same versioned RIG.
- Source-grounded by default: claims and relationships must point to real code and a specific commit.
- Honest uncertainty: distinguish extracted facts, inferred relationships, and generated narrative.
- Useful before perfect: return partial results when some files or framework patterns cannot be resolved.
- Safe by design: inspect untrusted source code; never execute repository code in the analysis pipeline.
- Progressive capability: ship a narrow, reliable first experience, while keeping the shared model extensible.

3. Problem statement

Developers joining a project, debugging an unfamiliar feature, reviewing a change, or preparing for an interview often need to reconstruct relationships scattered across routes, middleware, controllers, services, repositories, database calls, and external APIs. Reading files one by one is slow; generic summaries can omit control flow or hallucinate relationships; static diagrams quickly become stale.

Target user outcomes

- Reach a repository's analysis in one quick URL edit.
- Find a relevant entrypoint without knowing the codebase's internal naming.
- Follow an execution path and jump to the exact source lines supporting each step.
- Inspect a symbol's role and surrounding dependencies without manually searching the whole repository.
- Eventually compare structural changes across commits without independently rebuilding architecture diagrams.

4. Target users and primary use cases

Persona	Job to be done	Initial priority

Developer onboarding to an unfamiliar repo	Build a mental model quickly and find the important modules and entrypoints.	Primary
Full-stack engineer debugging a feature	Trace request-to-database or request-to-external-service behavior.	Primary
Code reviewer / maintainer	Understand the blast radius and connections of a symbol or module.	Secondary
Student / interview candidate	Explore a real-world codebase and explain how a feature works.	Secondary
Tech lead / architect	Understand coupling and how architecture changes over time.	Later; GitTime

Jobs to be done

- When I encounter an unfamiliar public repository, I want to open its GitHow page directly from the repository URL so I can start exploring without creating an account.
- When I want to understand a behavior, I want a source-linked flow so I can verify every step rather than trusting a black-box summary.
- When I select a function or class, I want its role and connections explained using the surrounding code as evidence.
- When I investigate project evolution, I want to compare meaningful architectural snapshots and see which commits introduced or removed relationships.

5. Product experience and information architecture

5.1 Primary entry: GitHub-domain replacement

For a public repository, the intended mental model is: github.com/owner/repo → githow.com/owner/repo. The destination must open that repository's GitHow experience directly—not a generic landing page that asks the user to paste the URL again.

- Preserve owner and repository path, with safe normalization for trailing slashes and supported GitHub URL variants.
- Resolve the repository and default branch to an immutable commit SHA for analysis and shareable results.
- If a cached analysis exists for the resolved SHA and analyzer version, open the repository view immediately.
- If analysis is missing, create or reuse a background job and show clear stage-level progress.
- If the repository is private, show a clear authorization-required state; private-repository support itself is post-MVP.
- Provide a clear path back to the original GitHub repository.

5.2 Secondary entry: GitHow homepage

- A prominent GitHub repository URL input.
- Submitting a valid public URL navigates to the same canonical repository route used by domain replacement.
- Validation errors explain what is wrong and how to fix it; arbitrary hosts are not accepted.

5.3 Repository workspace

A shared repository shell contains repository identity, commit/ref context, overview/navigation, the active mode, source viewer, and source-linked breadcrumbs. GitFlow, GitExplain, and GitTime are views within this shell.

Area	Purpose
Repository overview	Show repository metadata, detected languages/frameworks, module/dependency overview, analysis status and limitations.
GitFlow	Discover candidate flows, render the selected path, expand branches, and open source evidence.
GitExplain	Inspect a file/symbol's summary and graph neighborhood, including callers, callees and dependencies.
GitTime	Explore selected snapshots and structural diffs; enabled after the foundation is reliable.

Source panel	Display the correct file and highlighted line range for a selected node or edge.
Shareable URL	Preserve repository, commit, mode, flow/symbol/history target and relevant UI state.

6. Scope and release strategy

6.1 Product vision scope

- Public and private repository analysis, with appropriate authorization and isolation.
- Three views powered by a common RIG.
- Source-linked architecture, modules, dependencies, execution flows, symbol explanations and code evolution.
- Commit-pinned, shareable repository, flow, symbol and history links.
- Multiple language and framework plugins over time.
- Optional GitHub-native entry mechanisms such as a browser extension.

6.2 MVP scope

- Public GitHub repositories only; no login required.
- GitHub-domain replacement behavior and homepage URL submission both resolve to the same repository workspace.
- TypeScript/JavaScript analysis, initially prioritizing Express applications.
- Repository ingestion at a resolved commit SHA; manifest, size limits, cache lookup and job progress.
- RIG foundation with source references, provenance, confidence and analyzer versioning.
- GitFlow: route/entrypoint discovery, bounded traversal, source-linked interactive flow, expandable alternatives where supported.
- Thin GitExplain: selecting a graph node reveals its structural neighborhood and optional grounded summary.
- Repository overview, clear partial/unsupported analysis states, shareable commit-pinned links.
- GitTime data model/interface preparation only; no full timeline UI in MVP.

6.3 Explicitly out of MVP

- Private repository ingestion and GitHub App installation flow.
- GitHub browser extension or automatic browser rewriting; the domain-replacement URL is the core web experience, not an extension dependency.
- Full GitTime timeline, multi-snapshot graph diffing and entity lineage UI.
- Python and broad multi-language coverage.
- Repository code execution, dependency installation, running tests or dynamic tracing.
- Embeddings/vector search, collaboration, teams, comments, enterprise SSO, on-prem deployment, code editing or PR generation.
- Microservices, Kubernetes or a dedicated graph database solely for the MVP.

7. Functional requirements

Priority key: P0 = required for MVP; P1 = next meaningful release; P2 = later/future.

7.1 Entry, routing and repository resolution

FR-001 — Direct repository routing [P0]. A path in the form /{owner}/{repo} on the GitHow domain opens the matching repository workspace. The user must not be sent through the homepage first.

FR-002 — Homepage URL submission [P0]. The homepage accepts a public GitHub repository URL and navigates to the same canonical workspace.

FR-003 — URL validation [P0]. Accept supported github.com repository URLs, normalize harmless variants, and reject arbitrary hosts, malformed paths, and non-repository pages with a useful message.

FR-004 — Canonical identity [P0]. Resolve repository identity and the selected/default branch to a commit SHA; retain the SHA and analyzer version on the resulting snapshot.

FR-005 — Shareable deep links [P0]. Support canonical repository, pinned snapshot, flow and symbol routes; preserve relevant mode and selection state in the URL.

FR-006 — Private repository response [P0]. Detect inaccessible/private repositories and explain that authorization is not yet supported in MVP; do not misreport them as generic analysis failures.

7.2 Ingestion and analysis lifecycle

FR-010 — Public repository ingestion [P0]. Fetch a repository snapshot at a specific SHA and create a file manifest with paths, hashes, sizes and detected languages.

FR-011 — Cache reuse [P0]. Reuse a compatible completed analysis keyed by repository, commit SHA and analyzer version; prevent duplicate concurrent work for the same key.

FR-012 — Background processing [P0]. Run ingestion and analysis in background jobs so HTTP requests remain responsive.

FR-013 — Progress visibility [P0]. Expose understandable stages and progress states, including queued, fetching, inventory, parsing, graph construction, deriving views, complete and failed.

FR-014 — Partial results [P0]. Continue when individual files fail to parse where safe; surface incomplete-analysis warnings and affected file counts.

FR-015 — Retryability [P0]. Record stage-specific failures and support safe retry without corrupting or duplicating snapshots.

7.3 Repository Intelligence Graph

FR-020 — Shared graph [P0]. Persist normalized nodes and edges in PostgreSQL as the shared structural representation consumed by every mode.

FR-021 — Source references [P0]. Every source-backed node and relationship must carry or reference repository, commit SHA, file path and source range evidence where available.

FR-022 — Provenance and confidence [P0]. Label extracted facts, inferred relationships and LLM-generated narrative distinctly; attach confidence and evidence to edges.

FR-023 — Deterministic facts [P0]. The LLM must not create structural facts that are presented as extracted truth. Ambiguous resolutions must remain visibly uncertain.

FR-024 — Versioning [P0]. Store analyzer/version metadata and make analysis reproducible for the same commit and analyzer version.

FR-025 — Bounded graph queries [P0]. Graph traversal and API responses must have depth, fan-out, pagination and payload limits.

7.4 GitFlow

FR-030 — Entrypoint discovery [P0]. Find and display candidate application entrypoints/routes for supported frameworks.

FR-031 — Flow search and selection [P0]. Allow a user to select or search for a behavior using route, symbol, path or natural-language query; any LLM ranking is restricted to real candidates.

FR-032 — Ordered execution path [P0]. Render the best-supported path across relevant middleware, handler/controller, service, data-access, database or external-service nodes.

FR-033 — Evidence navigation [P0]. Selecting a flow step opens the exact supporting source file and line range at the analyzed commit.

FR-034 — Branch and uncertainty handling [P0]. Represent supported alternate branches as expandable alternatives and mark inferred/ambiguous edges clearly.

FR-035 — Flow sharing [P0]. A selected flow has a stable shareable URL pinned to the analyzed snapshot.

7.5 GitExplain

FR-040 — Node inspection [P0]. Clicking a supported file/symbol node opens its structural neighborhood and available source evidence.

FR-041 — Grounded explanation [P0]. Optional generated summaries must be based on a deterministic context bundle and provide citations that resolve to the supplied source/graph context.

FR-042 — Structural facts remain structural [P0]. Callers, callees, imports, routes and data/external interactions are rendered from the graph rather than paraphrased as unverified prose.

FR-043 — Graceful no-LLM mode [P0]. If the LLM provider fails or is unavailable, structural GitExplain and GitFlow views remain usable.

7.6 GitTime

FR-050 — Stable entity identity [P0]. The data model must support matching logical entities across commits and storing commit-to-file relationships.

FR-051 — Timeline and graph diff [P2]. Show sampled snapshots, added/removed/changed nodes and edges, and source-linked responsible commits.

FR-052 — Incremental history analysis [P2]. Reuse unchanged file analysis and limit full graph snapshots to selected meaningful commits.

7.7 Trust, privacy and abuse protection

FR-060 — No source execution [P0]. Never execute repository code, install dependencies, run build scripts or tests as part of analysis.

FR-061 — Safe archive handling [P0]. Protect against path traversal, symlinks, decompression bombs, oversized archives and excessive file counts.

FR-062 — Abuse controls [P0]. Rate-limit submission and bound repository size, file count, parsing time, memory and graph traversal.

FR-063 — Prompt-injection resistance [P0]. Treat repository text as untrusted data; LLM calls have no side-effect tools and outputs are schema- and citation-validated.

FR-064 — Privacy-ready schema [P0]. Represent repository visibility and ownership/tenant scope from the start even though private repositories are post-MVP.

FR-065 — No sensitive logs [P0]. Do not log source contents, access tokens or secrets; use identifiers, sizes, stage and request/job IDs.

8. Key user journeys

Journey A — URL replacement for a new public repository

1. User is on github.com/owner/repo.
2. User changes the domain to the GitHow domain while keeping the repository path.
3. GitHow validates and normalizes owner/repo, resolves the repository and commit SHA.
4. If a compatible snapshot is cached, the repository workspace opens; otherwise an analysis job starts and progress is displayed.

5. When ready, the user sees the overview and opens GitFlow.
6. The user selects a flow step and inspects the exact source lines behind it.

Journey B — Homepage paste

7. User opens the GitHow homepage.
8. User pastes a GitHub repository URL and submits.
9. The same validation and repository-resolution logic is used as the URL-replacement path.
10. User lands on the same canonical repository workspace, with cached or in-progress analysis.

Journey C — Understanding a flow

11. User opens the GitFlow view and sees candidate entrypoints.
12. User chooses a route or searches for a behavior.
13. GitFlow selects from actual graph candidates and assembles a bounded path.
14. The UI renders ordered steps, source evidence, and visible uncertainty/branches.
15. User opens a step's source reference and shares the commit-pinned flow URL if useful.

9. UX states and error behavior

State	Expected behavior
Invalid URL	Inline explanation of accepted GitHub URL shape; do not create a job.
Repository not found	Explain that the repository could not be found or accessed; offer URL correction.
Private/inaccessible repo	State that public repositories are supported in MVP and authorization is not yet available.
Queued / analyzing	Show stage, concise progress copy, and a way to wait or revisit the same URL.
Already analyzed	Open cached analysis without a redundant progress screen.
Unsupported language/framework	Show available overview/module/import information and explain which views are unavailable.
Partial parse failure	Show usable results plus a warning; do not silently claim full coverage.
LLM unavailable	Keep graph-backed navigation working; omit generated narrative or show a clear fallback.
Rate limited / repository too large	Explain the limit, avoid endless retries, and provide a clear next step.
Analysis failure	Show a user-safe error and retry option; keep internal stack traces out of the UI.

10. Non-functional requirements

Category	Requirement / target
Performance	Guidance target: typical small/medium public repository analysis completes in a few minutes or less; cached repository view opens in under 1 second where infrastructure permits; a cached flow renders in under 2 seconds after its request.
Reliability	Jobs are idempotent and retryable; failures are stage-attributed; partial results are preferred to total failure when safe.
Correctness	Every displayed flow step has a valid source reference; no dangling graph edges; provenance/confidence is preserved end to end.
Security	Parse-only analysis; constrained workers; safe URL allowlist; archive protections; bounded resources; rate limiting.
Privacy	Public analyses may use shared cache. Private data, once

	supported, must be tenant-scoped and excluded from shared source/LLM caches.
Accessibility	Keyboard-accessible navigation for key controls, readable contrast, and non-color-only uncertainty indicators.
Observability	Structured logs, job-stage duration, queue depth, cache hit rate, parse failures, unresolved-call ratio, flow production rate and API latency.
Maintainability	Typed shared contracts, modular code, golden fixtures, schema validation and automated tests for critical paths.

11. Success metrics

Instrument these from the earliest usable build. Initial numeric targets should be set after a baseline on a small golden corpus rather than guessed.

Metric	Definition	Why it matters
Direct-entry success	Share of valid public repository path visits that reach the intended repository workspace.	Validates the defining URL-first experience.
Analysis completion rate	Share of accepted repository jobs that complete or return useful partial results.	Measures operational reliability.
Time to first useful view	Time from URL submission/domain entry to overview or usable cached results.	Measures perceived speed.
Flow coverage	Share of supported fixture routes for which a source-linked flow is produced.	Measures the core product capability.
Flow evidence validity	Share of rendered flow steps whose source reference resolves to the correct commit and line range.	Measures trustworthiness.
Flow usefulness	User-rated usefulness or task completion on representative “how does X work?” tasks.	Measures product value, not just graph size.
Cache hit rate	Share of repository requests served from compatible cached analysis.	Improves speed and controls cost.
Return/share rate	Share of users who revisit or share a repository/flow link.	Signals repeat utility and distribution.
LLM cost per successful analysis	LLM spend attributable to a completed repository/flow, separated from deterministic processing.	Keeps economics visible.

12. Acceptance criteria for the first end-to-end release

- Changing github.com/owner/repo to the GitHow domain plus the same path opens the matching repository workspace directly.
- Pasting that same URL into the homepage lands on the same canonical workspace.
- The app validates URLs and rejects non-GitHub hosts without fetching arbitrary URLs.
- The system resolves and records a commit SHA and analyzer version for each completed snapshot.
- The same repository/SHA/analyzer version does not trigger redundant concurrent analysis jobs.
- A representative Express/TypeScript fixture produces a repository overview and at least one meaningful route-to-handler flow.
- Each rendered flow step opens the correct source file and line range for the pinned commit.
- Extracted facts and inferred relationships are distinguishable in the UI or accompanying evidence details.
- If an LLM call fails, the structural graph/flow remains available.
- Repository code is never executed; archive limits and resource limits are enforced.

- Automated tests cover URL parsing, commit resolution behavior, ingestion safety, job idempotency, graph invariants, source references and the main URL-entry journey.
- README and local development instructions explain how to run the web app, API, worker, PostgreSQL, queue and object storage locally.

13. Release roadmap

Phase	Outcome	Exit condition
0. Product & engineering foundation	Architecture and PRD aligned; repo structure, contracts, local services, CI and test conventions established.	A clean local setup and automated checks work.
1. Ingestion and direct entry	URL-first route, homepage paste, GitHub resolution, SHA-pinned snapshot, manifest, queue and progress.	A public repo can be submitted and safely ingested end to end.
2. Analysis and RIG	TS/JS + Express extraction, symbol/import resolution, normalized graph and validation.	Golden fixtures produce valid source-backed nodes and edges.
3. GitFlow	Entrypoint discovery, bounded traversal, flow API and interactive source-linked graph.	Representative routes produce verifiable flows.
4. GitExplain thin layer	Node inspection and grounded optional summaries.	A user can inspect a flow node and understand its connections.
5. Quality and public launch	Hardening, observability, caching, performance, share previews, abuse protection and docs.	Acceptance criteria and release checks pass.
6. GitExplain full	Richer data-flow, risk hints and symbol-level navigation.	Explanations remain evidence-backed on evaluation fixtures.
7. GitTime	History sampling, graph diffs, lineage and timeline UI.	Users can inspect meaningful structural changes across commits.
8. Expansion	Python, more frameworks, private repositories, GitHub App, extension, webhooks and optional search enhancements.	Each capability has isolated security, correctness and cost checks.

14. Dependencies, assumptions and open decisions

Assumptions

- GitHow is the umbrella product name; GitFlow, GitExplain and GitTime are views/modes within the platform.
- Public repository analysis is anonymous in MVP.
- The first implementation prioritizes TypeScript/JavaScript and Express, with the architecture allowing additional plugins later.
- The shared RIG and immutable commit SHA snapshots are foundational and must not be replaced with per-mode analysis.
- URL replacement is a first-class product requirement. The exact domain is subject to final domain/trademark checks and DNS/deployment setup.

Open decisions to resolve before or during implementation

- Final production domain and redirect/canonical URL policy, including owner/repo collisions with reserved routes.
- Repository size/file-count caps and acceptable analysis time, calibrated using representative public repositories.
- Initial golden fixture repositories and exact supported Express patterns.
- Which LLM provider/model is used for candidate ranking and narrative, with caching and cost budgets.
- Whether the MVP shows generated narrative by default or offers it as an enhancement after graph results load.

- Public snapshot retention policy and source-archive retention details.
- Final visual design and graph interaction details, validated against real flows rather than mock data alone.

15. Risks and mitigations

Risk	Impact	Mitigation
Incorrect or incomplete static analysis	Users lose trust if a flow looks definitive but is wrong.	Source evidence, confidence/provenance, ambiguity display, golden fixtures and graceful partial results.
Large or hostile repositories	Resource exhaustion or abuse.	Strict limits, safe archive extraction, timeouts, worker isolation and rate limits.
LLM hallucination or prompt injection	Incorrect explanations or unsafe behavior.	LLM selects/summarizes only from provided context; schema/citation validation; no side-effect tools; graph remains authoritative.
GitHub rate limits	Slow or failed repository resolution/ingestion.	ETag caching, request deduplication, backoff, platform GitHub App/token strategy when available.
Scope creep across three modes	MVP never becomes reliable.	Ship GitFlow first, keep GitExplain thin, reserve GitTime implementation for a later phase.
URL replacement path ambiguity	Direct entry accidentally routes to a generic page or conflicts with app routes.	Define reserved paths, test routing explicitly, and keep canonical repository URLs stable.
LLM and storage costs	Unsustainable cost per repository.	Cache by immutable SHA/context, cap work, track cost, and ensure core graph navigation works without LLM.

16. Product rules that must not be weakened

- The user must be able to enter through the repository URL path directly; homepage paste is a fallback, not a replacement for this experience.
- GitHow is one platform with one shared Repository Intelligence Graph—not three separately analyzed products.
- A source-backed relationship is not presented as a verified fact unless evidence supports it.
- Repository code is never executed during analysis.
- The UI must distinguish unsupported/partial analysis from a fully analyzed repository.
- GitFlow quality and source evidence are more important than the raw number of graph nodes or the number of supported languages.

Appendix A — Relationship to the architecture document

This PRD intentionally leaves technical implementation choices—monorepo structure, TypeScript, Next.js, API framework, PostgreSQL, pg-boss, object storage, tree-sitter, API contracts, worker architecture and detailed graph schema—to the Architecture Design Document. The PRD adds the product-level priority that the public GitHub-domain replacement path must be treated as the primary entry experience. The architecture should explicitly implement and test that routing behavior, while preserving its existing shared-RIG and commit-pinned design.